

Qualité de code en C++

Bonnes pratiques, robustesse et contexte embarqué

Document pédagogique complet

7 avril 2026

Table des matières

1	Introduction	5
1.1	Objectif du document	5
1.2	Criticité	5
1.3	Contenu du document	5
1.4	Approche pédagogique	5
2	Compilation stricte	6
2.1	Mauvais code	6
2.2	Bon code	6
2.3	Explication	6
3	Allocation dynamique interdite en contexte embarqué	7
3.1	Mauvais code	7
3.2	Bon code	7
3.3	Explication	7
4	Exceptions souvent interdites	8
4.1	Mauvais code	8
4.2	Bon code	8
4.3	Explication	8
5	Dépassement de tableau	9
5.1	Mauvais code	9
5.2	Bon code	9
5.3	Explication	9
6	Variables non initialisées	10
6.1	Mauvais code	10
6.2	Bon code	10
6.3	Explication	10
7	Déréférencement de pointeur nul	11
7.1	Mauvais code	11
7.2	Bon code	11
7.3	Explication	11
8	Référence pendante	12
8.1	Mauvais code	12
8.2	Bon code	12
8.3	Explication	12

9	Use-after-scope	13
9.1	Mauvais code	13
9.2	Bon code	13
9.3	Explication	13
10	Overflow signé	14
10.1	Mauvais code	14
10.2	Bon code	14
10.3	Explication	14
11	Division par zéro	15
11.1	Mauvais code	15
11.2	Bon code	15
11.3	Explication	15
12	Conversions signé / non signé	16
12.1	Mauvais code	16
12.2	Bon code	16
12.3	Explication	16
13	Narrowing et troncature	17
13.1	Mauvais code	17
13.2	Bon code	17
13.3	Explication	17
14	Double promotion	18
14.1	Mauvais code	18
14.2	Bon code	18
14.3	Explication	18
15	Cast C-style dangereux	19
15.1	Mauvais code	19
15.2	Bon code	19
15.3	Explication	19
16	Affectation au lieu de comparaison	20
16.1	Mauvais code	20
16.2	Bon code	20
16.3	Explication	20
17	Switch avec fallthrough involontaire	21
17.1	Mauvais code	21
17.2	Bon code	21
17.3	Explication	21
18	Switch non exhaustif sur enum	22
18.1	Mauvais code	22
18.2	Bon code	22
18.3	Explication	23

19 Accolades manquantes	24
19.1 Mauvais code	24
19.2 Bon code	24
19.3 Explication	24
20 Comparaison flottante directe	25
20.1 Mauvais code	25
20.2 Bon code	25
20.3 Explication	25
21 Commentaires trompeurs	26
21.1 Mauvais code	26
21.2 Bon code	26
21.3 Explication	26
22 Convention de nommage	27
22.1 Mauvais code	27
22.2 Bon code	27
22.3 Explication	27
23 Fonctions trop longues	28
23.1 Mauvais code	28
23.2 Bon code	28
23.3 Explication	28
24 Nombres magiques	29
24.1 Mauvais code	29
24.2 Bon code	29
24.3 Explication	29
25 Valeur de retour ignorée	30
25.1 Mauvais code	30
25.2 Bon code	30
25.3 Explication	30
26 Const-correctness	31
26.1 Mauvais code	31
26.2 Bon code	31
26.3 Explication	31
27 Concurrence : data race	32
27.1 Mauvais code	32
27.2 Bon code	32
27.3 Explication	32
28 Destructeur non virtuel	33
28.1 Mauvais code	33
28.2 Bon code	33
28.3 Explication	33

29 Object slicing	34
29.1 Mauvais code	34
29.2 Bon code	34
29.3 Explication	35
30 Buffer overflow via sprintf	36
30.1 Mauvais code	36
30.2 Bon code	36
30.3 Explication	36
31 Variables d'interruption et volatile	37
31.1 Mauvais code	37
31.2 Bon code	37
31.3 Explication	37
32 Initialisation globale risquée	38
32.1 Mauvais code	38
32.2 Bon code	38
32.3 Explication	38
33 Alignement mémoire et padding	39
33.1 Mauvais code	39
33.2 Bon code	39
33.3 Explication	39
34 Conclusion	40

Chapitre 1

Introduction

1.1 Objectif du document

Ce document a pour objectif de présenter les bonnes pratiques essentielles en langage C++ à travers une approche comparative entre **mauvais** et **bon code**. Il vise à améliorer la qualité, la robustesse et la maintenabilité des programmes en mettant en évidence les erreurs courantes et leurs corrections.

1.2 Criticité

Le langage C++ est puissant, expressif et performant, mais il reste exigeant. Une mauvaise pratique peut rapidement conduire à :

- des comportements indéfinis
- des erreurs mémoire
- des bugs liés aux conversions, aux durées de vie ou au polymorphisme
- des défauts difficiles à détecter et à corriger

Adopter des règles strictes permet de mieux maîtriser la complexité du langage et de produire un code fiable, lisible et adapté à un usage industriel ou embarqué.

1.3 Contenu du document

Ce document couvre les aspects fondamentaux de la qualité en C++ :

- Compilation stricte et gestion des warnings
- Bon usage des types, conversions et casts
- Gestion sûre des objets, pointeurs, références et durées de vie
- Prévention des comportements indéfinis et des erreurs mémoire
- Écriture de code lisible, structuré et maintenable
- Maîtrise des bonnes pratiques liées à la robustesse, à la concurrence et au contexte embarqué

1.4 Approche pédagogique

Chaque chapitre suit une structure simple et efficace :

- un exemple de **mauvais code**
- une version corrigée (**bon code**)
- une **explication** claire du problème et de la solution

Cette approche permet de comprendre rapidement les erreurs fréquentes et d'adopter les bons réflexes en situation réelle.

Chapitre 2

Compilation stricte

2.1 Mauvais code

```
#include <cstdio>

void log_u32(unsigned v) {
    std::printf("%d\n", v);
}
```

2.2 Bon code

```
#include <cstdio>

void log_u32(unsigned v) {
    std::printf("%u\n", v);
}
```

2.3 Explication

La compilation stricte détecte très tôt les erreurs de format, les conversions implicites dangereuses, les oublis de `break`, les masquages de variables ou encore les destructeurs non virtuels. Une base de compilation recommandée est :

```
g++ -std=c++20 -O2 -g -Wall -Wextra -Wpedantic -Werror \
    -Wshadow -Wconversion -Wsign-conversion \
    -Wnull-dereference -Wdouble-promotion -Wformat=2 \
    -Wimplicit-fallthrough -Wold-style-cast -Wnon-virtual-dtor \
    fichier.cpp -o app
```

L'objectif est d'obtenir zéro warning.

Chapitre 3

Allocation dynamique interdite en contexte embarqué

3.1 Mauvais code

```
int* p = new int(42);  
delete p;
```

3.2 Bon code

```
#include <array>  
#include <cstdint>  
  
static std::array<int, 4> buffer{};  
static std::uint32_t value = 42;
```

3.3 Explication

En embarqué critique, on évite souvent le heap à cause de la fragmentation, du manque de déterminisme et des échecs d'allocation au runtime. On préfère des structures statiques ou de taille fixe.

Chapitre 4

Exceptions souvent interdites

4.1 Mauvais code

```
int f() {  
    throw 42;  
}
```

4.2 Bon code

```
enum class Status {  
    Ok,  
    Error  
};  
  
Status do_work() noexcept {  
    return Status::Ok;  
}
```

4.3 Explication

Dans beaucoup de projets embarqués, les exceptions sont désactivées pour réduire la taille binaire et garantir un comportement plus déterministe. On utilise alors des codes retour explicites et `noexcept`.

Chapitre 5

Dépassement de tableau

5.1 Mauvais code

```
int main() {  
    int a[3] = {1, 2, 3};  
    a[3] = 99;  
}
```

5.2 Bon code

```
#include <array>  
  
int main() {  
    std::array<int, 3> a{1, 2, 3};  
    a.at(2) = 99;  
}
```

5.3 Explication

Accéder hors des bornes d'un tableau provoque un comportement indéfini. Les conteneurs standards comme `std::array` permettent des accès plus sûrs, notamment avec `at()`.

Chapitre 6

Variables non initialisées

6.1 Mauvais code

```
#include <stdio>

int main() {
    int x;
    std::printf("%d\n", x);
}
```

6.2 Bon code

```
#include <stdio>

int main() {
    int x = 0;
    std::printf("%d\n", x);
}
```

6.3 Explication

Lire une variable non initialisée donne une valeur indéterminée. C'est une source fréquente de bugs aléatoires.

Chapitre 7

Déréférencement de pointeur nul

7.1 Mauvais code

```
int read(const int* p) {  
    return *p;  
}
```

7.2 Bon code

```
#include <optional>  
  
[[nodiscard]] std::optional<int> read(const int* p) {  
    if (!p) {  
        return std::nullopt;  
    }  
    return *p;  
}
```

7.3 Explication

Déréférencer `nullptr` provoque un comportement indéfini, souvent un crash. Il faut vérifier les pointeurs reçus avant usage.

Chapitre 8

Référence pendante

8.1 Mauvais code

```
const int& bad_ref() {  
    int x = 42;  
    return x;  
}
```

8.2 Bon code

```
int good_value() {  
    int x = 42;  
    return x;  
}
```

8.3 Explication

Retourner une référence sur une variable locale est invalide dès la sortie de la fonction. Il faut retourner une valeur ou utiliser une durée de vie adaptée.

Chapitre 9

Use-after-scope

9.1 Mauvais code

```
int* g_ptr = nullptr;

void f() {
    int x = 10;
    g_ptr = &x;
}

int main() {
    f();
    return *g_ptr;
}
```

9.2 Bon code

```
void set_value(int& out) {
    out = 10;
}

int main() {
    int x = 0;
    set_value(x);
    return x;
}
```

9.3 Explication

Conserver l'adresse d'une variable locale au-delà de sa portée mène à un comportement indéfini. Il faut laisser l'appelant gérer la durée de vie des objets qu'il fournit.

Chapitre 10

Overflow signé

10.1 Mauvais code

```
#include <climits>

int main() {
    int x = INT_MAX;
    x += 1;
}
```

10.2 Bon code

```
#include <limits>
#include <optional>

[[nodiscard]] std::optional<int> add_safe(int a, int b) {
    if (b > 0 && a > std::numeric_limits<int>::max() - b) {
        return std::nullopt;
    }
    if (b < 0 && a < std::numeric_limits<int>::min() - b) {
        return std::nullopt;
    }
    return a + b;
}
```

10.3 Explication

Le dépassement de capacité d'un entier signé est un comportement indéfini en C++. Il faut contrôler les bornes avant l'opération.

Chapitre 11

Division par zéro

11.1 Mauvais code

```
int main() {  
    int a = 10;  
    int b = 0;  
    return a / b;  
}
```

11.2 Bon code

```
#include <optional>  
  
[[nodiscard]] std::optional<int> div_checked(int a, int b) {  
    if (b == 0) {  
        return std::nullopt;  
    }  
    return a / b;  
}
```

11.3 Explication

Diviser un entier par zéro est un comportement indéfini. Le contrôle préalable est obligatoire.

Chapitre 12

Conversions signé / non signé

12.1 Mauvais code

```
#include <cstdint>

int main() {
    int i = -1;
    std::size_t n = static_cast<std::size_t>(i);
    (void)n;
}
```

12.2 Bon code

```
#include <optional>
#include <cstdint>

[[nodiscard]] std::optional<std::size_t> to_size(int i) {
    if (i < 0) {
        return std::nullopt;
    }
    return static_cast<std::size_t>(i);
}
```

12.3 Explication

Un entier négatif converti en type non signé devient une très grande valeur. Il faut vérifier le domaine de validité avant conversion.

Chapitre 13

Narrowing et troncature

13.1 Mauvais code

```
#include <cstdint>

std::uint8_t v = 300;
```

13.2 Bon code

```
#include <optional>
#include <cstdint>

[[nodiscard]] std::optional<std::uint8_t> to_u8(int value) {
    if (value < 0 || value > 255) {
        return std::nullopt;
    }
    return static_cast<std::uint8_t>(value);
}
```

13.3 Explication

Affecter une valeur trop grande dans un type plus petit tronque l'information. Cette troncature doit être évitée ou validée explicitement.

Chapitre 14

Double promotion

14.1 Mauvais code

```
float x = 1.0f;  
float y = x + 0.1;
```

14.2 Bon code

```
float x = 1.0f;  
float y = x + 0.1f;
```

14.3 Explication

Un littéral décimal comme 0.1 est un **double** par défaut. Cela force une promotion implicite qui peut dégrader les performances ou provoquer des warnings.

Chapitre 15

Cast C-style dangereux

15.1 Mauvais code

```
double d = 3.14;  
int x = (int&)d;
```

15.2 Bon code

```
double d = 3.14;  
int x = static_cast<int>(d);
```

15.3 Explication

Les casts C-style masquent plusieurs types de conversion possibles. Les casts C++ rendent l'intention explicite et permettent une lecture plus sûre.

Chapitre 16

Affectation au lieu de comparaison

16.1 Mauvais code

```
bool ok = false;

if (ok = true) {
}
```

16.2 Bon code

```
bool ok = false;

if (ok) {
}
```

16.3 Explication

Utiliser = dans une condition modifie la variable. Cette erreur classique peut être détectée avec les warnings de compilation.

Chapitre 17

Switch avec fallthrough involontaire

17.1 Mauvais code

```
int f(int x) {  
    switch (x) {  
        case 0:  
            doA();  
        case 1:  
            doB();  
            break;  
        default:  
            return 0;  
    }  
    return 1;  
}
```

17.2 Bon code

```
int f(int x) {  
    switch (x) {  
        case 0:  
            doA();  
            break;  
        case 1:  
            doB();  
            break;  
        default:  
            return 0;  
    }  
    return 1;  
}
```

17.3 Explication

Sans `break`, l'exécution continue dans le cas suivant. Cela peut être voulu, mais dans ce cas il faut le documenter explicitement.

Chapitre 18

Switch non exhaustif sur enum

18.1 Mauvais code

```
enum class State {  
    Idle,  
    Run,  
    Error  
};  
  
void handle(State s) {  
    switch (s) {  
        case State::Idle:  
            break;  
        case State::Run:  
            break;  
    }  
}
```

18.2 Bon code

```
enum class State {  
    Idle,  
    Run,  
    Error  
};  
  
void handle(State s) {  
    switch (s) {  
        case State::Idle:  
            break;  
        case State::Run:  
            break;  
        case State::Error:  
            break;  
    }  
}
```

18.3 Explication

Oublier un état dans un `switch` sur une énumération crée un bug silencieux. Chaque valeur doit être traitée explicitement ou via un `default` justifié.

Chapitre 19

Accolades manquantes

19.1 Mauvais code

```
if (ok)
    doA();
    doB();
```

19.2 Bon code

```
if (ok) {
    doA();
    doB();
}
```

19.3 Explication

Sans accolades, l'indentation peut devenir trompeuse. Il faut préférer des blocs explicites, même pour un seul statement.

Chapitre 20

Comparaison flottante directe

20.1 Mauvais code

```
float a = 0.1f + 0.2f;

if (a == 0.3f) {
}
```

20.2 Bon code

```
#include <cmath>

bool nearly_equal(float a, float b, float eps) {
    return std::fabs(a - b) <= eps;
}
```

20.3 Explication

Les flottants sont approximatifs. Une comparaison directe avec `==` est donc fragile dans la plupart des cas.

Chapitre 21

Commentaires trompeurs

21.1 Mauvais code

```
#include <array>
#include <cstdint>

// Allocate 128 bytes of RAM
std::array<std::uint8_t, 14> txBuffer;
```

21.2 Bon code

```
#include <array>
#include <cstdint>

// txBuffer contient le message a emettre (14 octets fixes).
std::array<std::uint8_t, 14> txBuffer{};
```

21.3 Explication

Un commentaire faux ou imprécis nuit à la compréhension. Un bon commentaire explique la contrainte, l'intention ou la raison métier.

Chapitre 22

Convention de nommage

22.1 Mauvais code

```
int MaxValue = 100;
int max_value = 200;
int MAXVALUE = 300;

void DoStuff();
void do_stuff();
```

22.2 Bon code

```
constexpr int kMaxValue = 100;
int current_value = 0;

struct PacketHeader {
    int id;
};

void process_packet(const PacketHeader& h);
```

22.3 Explication

Une convention cohérente améliore fortement la lisibilité. Il faut définir une règle simple et la respecter dans tout le projet.

Chapitre 23

Fonctions trop longues

23.1 Mauvais code

```
void handle_request(Request r) {  
    /* parse + validate + auth + log + business + format */  
}
```

23.2 Bon code

```
Response handle_request(const Request& r) {  
    const auto parsed = parse(r);  
  
    if (!validate(parsed)) {  
        return error("bad request");  
    }  
  
    const auto user = authenticate(parsed);  
    return execute_business(user, parsed);  
}
```

23.3 Explication

Une fonction qui fait trop de choses devient difficile à tester, relire et maintenir. Il faut découper les responsabilités.

Chapitre 24

Nombres magiques

24.1 Mauvais code

```
int timeout_ms = 1500;
int retries = 4;

if (retries > 3) {
    abort();
}
```

24.2 Bon code

```
constexpr int kTimeoutMs = 1500;
constexpr int kMaxRetries = 3;

int retries = 4;
int timeout_ms = kTimeoutMs;

if (retries > kMaxRetries) {
    abort();
}
```

24.3 Explication

Les valeurs magiques rendent le code opaque. Les constantes nommées rendent l'intention explicite.

Chapitre 25

Valeur de retour ignorée

25.1 Mauvais code

```
struct Frame {
    int id;
};

bool send_frame(const Frame& f);

void task(const Frame& f) {
    send_frame(f);
}
```

25.2 Bon code

```
struct Frame {
    int id;
};

[[nodiscard]] bool send_frame(const Frame& f);

void task(const Frame& f) {
    if (!send_frame(f)) {
        /* gerer l'erreur */
    }
}
```

25.3 Explication

Ignorer une valeur de retour masque une erreur potentielle. `[[nodiscard]]` aide à imposer cette discipline.

Chapitre 26

Const-correctness

26.1 Mauvais code

```
#include <array>

int sum(std::array<int,3>& a) {
    return a[0] + a[1] + a[2];
}
```

26.2 Bon code

```
#include <array>

int sum(const std::array<int,3>& a) {
    return a[0] + a[1] + a[2];
}
```

26.3 Explication

Une API qui n'a pas besoin de modifier son argument doit l'exprimer avec `const`. Cela clarifie le contrat et élargit les cas d'usage.

Chapitre 27

Concurrence : data race

27.1 Mauvais code

```
#include <thread>

int x = 0;

void f() {
    for (int i = 0; i < 100000; i++) {
        x++;
    }
}
```

27.2 Bon code

```
#include <thread>
#include <atomic>

std::atomic<int> x{0};

void f() {
    for (int i = 0; i < 100000; i++) {
        x.fetch_add(1);
    }
}
```

27.3 Explication

Deux threads qui modifient la même variable sans synchronisation créent une course de données. Le résultat devient non déterministe.

Chapitre 28

Destructeur non virtuel

28.1 Mauvais code

```
struct Base {  
    ~Base() = default;  
};  
  
struct Der : Base {  
    ~Der() = default;  
};
```

28.2 Bon code

```
struct Base {  
    virtual ~Base() = default;  
};  
  
struct Der : Base {  
    ~Der() override = default;  
};
```

28.3 Explication

Si une classe est utilisée polymorphiquement, son destructeur doit être virtuel. Sinon la destruction via pointeur de base est dangereuse.

Chapitre 29

Object slicing

29.1 Mauvais code

```
#include <cstdio>

struct Base {
    virtual ~Base() = default;
    virtual void f() { std::printf("B\n"); }
};

struct Der : Base {
    void f() override { std::printf("D\n"); }
    int x = 1;
};

int main() {
    Der d;
    Base b = d;
    b.f();
}
```

29.2 Bon code

```
#include <cstdio>

struct Base {
    virtual ~Base() = default;
    virtual void f() { std::printf("B\n"); }
};

struct Der : Base {
    void f() override { std::printf("D\n"); }
    int x = 1;
};

int main() {
    Der d;
    Base& b = d;
    b.f();
}
```

29.3 Explication

Copier un objet dérivé dans un objet base par valeur supprime la partie dérivée. Il faut passer par référence ou pointeur quand on veut conserver le polymorphisme.

Chapitre 30

Buffer overflow via sprintf

30.1 Mauvais code

```
#include <cstdio>

int main() {
    char buf[8];
    std::sprintf(buf, "value=%d", 123456);
}
```

30.2 Bon code

```
#include <cstdio>
#include <cstddef>

bool make(char* out, std::size_t cap, int v) {
    int n = std::snprintf(out, cap, "value=%d", v);
    return n >= 0 && static_cast<std::size_t>(n) < cap;
}
```

30.3 Explication

`sprintf` n'impose aucune limite de taille. `snprintf` permet d'éviter les dépassements de tampon.

Chapitre 31

Variables d'interruption et volatile

31.1 Mauvais code

```
bool data_ready = false;

void UART_ISR() {
    data_ready = true;
}

int main() {
    while (!data_ready) {
    }
}
```

31.2 Bon code

```
volatile bool data_ready = false;

void UART_ISR() {
    data_ready = true;
}

int main() {
    while (!data_ready) {
    }
}
```

31.3 Explication

Une variable modifiée par interruption peut être optimisée à tort par le compilateur. Dans ce cas, `volatile` indique qu'il faut relire la valeur en mémoire.

Chapitre 32

Initialisation globale risquée

32.1 Mauvais code

```
/* A.cpp */
HardwareTimer timerA;

/* B.cpp */
TimerManager manager(timerA);
```

32.2 Bon code

```
constinit int g_hardware_config = 42;

HardwareTimer& get_timer() {
    static HardwareTimer timer;
    return timer;
}
```

32.3 Explication

L'ordre d'initialisation des objets globaux répartis dans plusieurs unités de traduction n'est pas maîtrisé. Le pattern *construct on first use* ou **constinit** réduit ce risque.

Chapitre 33

Alignement mémoire et padding

33.1 Mauvais code

```
#include <cstdint>

struct HardwareReg {
    std::uint8_t flag;
    std::uint32_t data;
};
```

33.2 Bon code

```
#include <cstdint>

struct __attribute__((packed)) HardwareRegPacked {
    std::uint8_t flag;
    std::uint32_t data;
};

static_assert(sizeof(HardwareRegPacked) == 5);
```

33.3 Explication

Le compilateur peut insérer des octets de bourrage dans une structure. Quand on mappe une structure sur du matériel ou un protocole, il faut maîtriser sa taille exacte.

Chapitre 34

Conclusion

Un code C++ de qualité professionnelle en approche DevSecOps repose sur :

- une compilation stricte sans warning ;
- une gestion rigoureuse des conversions, de la mémoire et des durées de vie ;
- des API explicites avec `const`, `[[nodiscard]]` et `noexcept` ;
- une attention particulière au polymorphisme, à la concurrence et à la robustesse ;
- un style cohérent et une structuration claire des responsabilités ;
- une adaptation aux contraintes embarquées quand le contexte l'exige.

Ces pratiques permettent de produire un code plus fiable, plus lisible, plus maintenable et plus sûr dans la durée.